

Assignment 4: Enterprise GenAI Agent (Text-to-SQL & Forecasting)

Project Overview

Build a production-grade **Data Intelligence Agent** for the "TheLook E-commerce" dataset (<https://console.cloud.google.com/marketplace/product/bigquery-public-data/thelook-ecommerce?project=s2s-459305>).

This project requires an **"Evaluation First" mindset**. You will not just build a chatbot; you will build a measurable system. You will start by exploring data and defining "What does good look like?" (Golden Dataset) before writing complex agent logic.

The system will use `google agent development kit` for orchestration, **Gemini** models for reasoning, and **Toolbox MCP** for standardized tool access. It must support **Text-to-SQL**, **Time-Series Forecasting**, and **On-Demand Visualization**.

Technology Stack

- **Orchestration:** google agent development kit (workflows)
- **LLM:** Gemini Flash and Pro (via Vertex AI or Google AI Studio)
- **Tooling Standard:** Model Context Protocol (MCP) using **Toolbox**
- **Forecasting:** `AI.Forecast` (via MCP / BigQuery ML)
- **Observability & Eval:** Arize Phoenix (Implemented from Day 1 of dev)
- **Backend:** FastAPI
- **Database:** MongoDB (Chat History, User Data)
- **Frontend:** Custom AI-Generated UI ("Vibecoding") with Chart.js
- **Infrastructure:** Docker, Google Cloud Run

Week 1: Data Strategy & Evaluation Foundation

Learning Objectives

Before writing a single line of agent code, you must master the data and define success. A great AI engineer understands the data domain first.

Tasks

Day 1-3: Deep Dive Data Exploration

1. **Exploratory Data Analysis (EDA):**
 - Access `bigquery-public-data.thelook_ecommerce`.
 - Manually write SQL to understand the schema. Identify the correct tables for:
 - Sales Revenue (Orders vs Order Items)

- User Growth (Users table)
- Product Inventory.
- 2. **Data Strategy Doc:**
 - Create a simple document listing the **5 Key Metrics** the bot must be able to calculate accurately.

Day 4-5: Creating the Golden Dataset

1. **The Golden Dataset (Crucial):**
 - Create a dataset of **20-30 QA pairs** in CSV/JSON format.
 - **Structure:**
 - *Natural Language Question:* "How many sneakers were sold in 2023?"
 - *Expected SQL:* `SELECT count(*) FROM ... WHERE category='Sneakers' AND year=2023`
 - *Expected Answer:* The actual number (ground truth).
 - *Why?* This dataset will be your "Unit Test" for the AI. You cannot proceed to Week 2 without this.

Deliverables

- Data Strategy Document.
- Golden Dataset (CSV/JSON) with 20+ pairs.

Week 2: Architecture, Observability & Core SQL

Learning Objectives

Design the system architecture, set up observability (Arize Phoenix), and build the core Text-to-SQL engine.

Tasks

Day 1-2: Architecture & Observability

1. **Setup Arize Phoenix:**
 - Install Arize Phoenix locally.
 - Configure your environment so that **all** LLM traces are visible in the Phoenix UI.
2. **Architecture Proposal:**
 - **Task:** Design the google agent development kit Workflow architecture.
 - **Constraint:** Do not follow a fixed template. Determine how to route user queries and how to handle tool execution.
 - **Review:** Present your architecture diagram to your mentor. Explain *why* you chose this structure (e.g., specific routing logic).

Day 3-5: The SQL Workflow (GenAI Core)

1. **Toolbox MCP Integration:**
 - Connect google agent development kit to the Toolbox MCP server.
 - Enable BigQuery tools to fetch table schemas.
2. **SQL Generation Logic:**
 - Implement the google agent development kit workflow branch that handles data retrieval.
 - **Robustness Loop:** Implement a self-correction step. If the generated SQL fails in BigQuery, the error must be fed back to the LLM to generate a fixed query automatically.

Deliverables

- Architecture Diagram (Approved).
- Arize Phoenix running with traces.
- Functional Text-to-SQL Agent (tested against 5 questions from Golden Dataset).

Week 3: Advanced Capabilities (Forecasting & Visualization)

Learning Objectives

Expand the agent's capabilities to handle time-series forecasting and on-demand visualization, then evaluate the performance.

Tasks

Day 1-2: Forecasting Workflow

1. **Forecasting Logic:**
 - Implement a workflow branch for predictive questions (e.g., *"Forecast sales for next month"*).
 - Use the `AI.Forecast` MCP tool (or BigQuery ML wrapper).
 - Ensure the LLM explains the forecast results clearly in natural language.

Day 3-4: The "Visualize" Workflow

1. **Data Structuring:**
 - The backend should not return an image. It must return a structured response: `can_visualize: true` (if data is suitable for a chart).
2. **Visualization Endpoint:**
 - Create a handler that generates **Chart.js compatible JSON** (labels, datasets, colors) based on the query result.

- *Note:* This is only triggered when the user explicitly requests it.

Day 5: Automated Evaluation

1. **Run the Benchmark:**
 - Run your **Golden Dataset** (from Week 1) through your full agent.
2. **LLM-as-a-Judge:**
 - Use a separate LLM call to grade the responses.
 - Analyze failures in Arize Phoenix (e.g., "Why did it fail to forecast?").

Deliverables

- Working Forecasting Agent.
- Visualization logic ready (Backend).
- Evaluation Report (Accuracy % on Golden Dataset).

Week 4: Backend & Database Integration

Learning Objectives

Wrap your intelligent workflows into a robust API service using FastAPI and MongoDB.

Tasks

Day 1-3: FastAPI Implementation

1. **API Endpoints:**
 - `POST /chat`: Accepts message, returns answer + `can_visualize` flag.
 - `POST /visualize`: Generates chart configuration for the current context.
2. **Streaming:**
 - Implement response streaming so the user sees the answer being typed out in real-time.

Day 4-5: MongoDB Persistence

1. **Chat History:**
 - Store conversation threads in MongoDB.
 - Ensure the agent has "Memory" (can answer follow-up questions).
2. **Session Management:**
 - Design schema to support multiple concurrent user sessions.

Deliverables

- Complete FastAPI backend.
- MongoDB connected with chat history storage.

Week 5: Frontend "Vibecoding"

Learning Objectives

Rapidly build a modern, interactive UI using AI-assisted coding tools.

Tasks

Day 1-3: UI Construction

1. **Vibecoding:**
 - Use AI tools (Cursor, v0.dev, etc.) to generate a React/Next.js or clean HTML/JS interface.
 - Focus on clean UX for chat bubbles and markdown rendering.

Day 4-5: Interactive Visualization

1. **The "Visualize" Button:**
 - Implement the logic: *If* backend says `can_visualize: true`, show a  "Show Chart" button.
2. **Chart.js Rendering:**
 - On button click, fetch the JSON and render a beautiful Chart.js graph inside the chat stream.

Deliverables

- Functional Chat UI.
- Dynamic Chart rendering on button click.

Week 6: User Authentication, Deployment & Final Polish

Learning Objectives

Secure the application and deploy the complete stack to the cloud.

Tasks

Day 1-3: Authentication (The Final Layer)

1. **JWT Auth:**
 - Now that the core product works, add `User` models and JWT authentication.

- Create Login/Signup pages.
- Ensure users can only access *their own* chat history.

Day 4-5: Cloud Deployment

1. Dockerization:

- Create Dockerfiles for:
 1. Toolbox MCP Server.
 2. FastAPI Backend (with LlamaIndex).
 3. Frontend UI.

2. Cloud Run:

- Deploy all services to Google Cloud Run.
- Configure secure networking between services.

Deliverables

- Live, secured application URL.
- Final source code repository.
- Video Demo covering: Auth, SQL Query, Forecasting, and Chart Visualization.

Assessment Criteria

- **Evaluation First:** Did you actually use the Golden Dataset to improve the agent, or did you just build blindly?
- **Architecture Defensibility:** Can you clearly explain *why* you designed the agent workflow this way?
- **Forecasting Accuracy:** Does the forecasting tool provide reasonable future predictions?
- **Code Quality:** Is the GenAI logic decoupled from the web server logic?